

HAPIQ: HIERARCHICAL ADAPTIVE-PRECISION INFERENCE FOR LARGE LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

We present HAPIQ, an open-source inference framework that cuts memory, latency, and energy for large-language-model (LLM) serving by assigning arithmetic precision on a per-token basis and by extending the same mechanism to the key-value (KV) cache. Prior adaptive-precision work limits decisions to the weight tensors or to one choice per layer, leaving up to half of the achievable savings unused. HAPIQ closes this gap through three innovations: (i) a quad-path bit-plane layout that stores both weights and KV tensors as an INT3 base plus optional single-bit residual planes, enabling constant-time upgrades or downgrades to INT4, INT8, FP8, or FP16; (ii) HSE-3, a 160 k-parameter hierarchical saliency engine that fuses self-attention logits, discourse-burst cues, and retrieval scores to emit 2-bit precision codes in under $2\ \mu\text{s}$; and (iii) the first public Triton kernel that executes INT3, INT4/8, FP8, and FP16 matrix multiplies concurrently. Evaluations on Llama-3-8B/70B, Mistral-7B, and Phi-3-Mini over MMLU, GSM8K, HellaSwag, LongFormQA-32 k, and streaming chat show that HAPIQ delivers 35-45 % higher throughput than uniform INT8, shrinks the KV cache by $2.2\times$, and lowers energy per token by 11 %, all with $< 0.25\%$ accuracy loss. Against HALO-Q v2 the speed-up remains 18-25 % on contexts $\geq 8\text{ k}$. All code, Triton kernels, and Docker recipes are released under Apache-2.

1 INTRODUCTION

Large language models (LLMs) drive modern search, coding assistants, and conversational agents, yet their inference cost grows with both parameter count and context length. Two bottlenecks dominate current deployments. First, the key-value (KV) cache can exceed 50 GB on 32 k-token prompts, pressuring GPU memory. Second, the mixed workloads typical of data-centre serving expose highly variable power and bandwidth head-room, so any static quantisation wastes resources.

Existing quantisation research addresses only fragments of this problem. Vector-wise INT8 with 16-bit outlier channels enables post-training deployment but fixes one precision per layer ?. Any-Precision LLM overlays multiple bit-width checkpoints but ignores KV tensors and runtime feedback ?. Consequently, 30-60 % of potential memory and compute savings remain unrealised.

We introduce Hierarchical Adaptive-Precision Inference for LLMs (HAPIQ), the first system to: 1. store weights and KV tensors in a common up-/down-gradable bit-plane format, 2. predict token-level saliency fast enough to steer precision in real time, and 3. react to power or congestion spikes within microseconds.

The design aligns storage with the four arithmetic paths of Blackwell-class GPUs—INT3, INT4/8, FP8, FP16—and schedules them concurrently inside a fused Triton kernel. The central challenge is that saliency estimation, precision switching, and heterogeneous GEMMs must finish within the token budget ($\approx 20\ \mu\text{s}$ at 50 tok s^{-1}) while keeping misprediction roll-backs below 5 %.

We solve this with: (i) quad-path bit-plane storage; (ii) HSE-3, a three-level saliency network that combines self-regard, burst detection, and retrieval signals; (iii) the first public INT3 GEMM for GPUs; and (iv) a dual-loop resource controller tuned by offline reinforcement learning.

1.1 KEY CONTRIBUTIONS

- **Unified bit-plane layout** supporting constant-time INT3→FP16 upgrades for both weights and KV tensors.
- **HSE-3** a 160 k-parameter saliency engine with $\leq 2 \mu\text{s}$ overhead.
- **Open INT3 GEMM** kernel released in Triton.
- **Precision-adaptive KV streaming** that halves peak KV memory.
- **Power- and bandwidth-aware scheduler** that cuts energy per token by 11 % under a 350 W cap.
- **Reproducible OSS stack** integrating PyTorch, Triton, and Docker under Apache-2.

The remainder of the paper details related work, background, method, experimental protocol, empirical results, and future directions.

2 RELATED WORK

Inference-time quantisation. LLM.int8() isolates outlier channels via mixed 8/16-bit arithmetic ?. Scaling-law analyses suggest 4-bit is often optimal for zero-shot accuracy ?, while AdaDim mitigates activation outliers with per-input-channel grouping ? and QuIP achieves 2-bit weights through incoherence processing ?. All choose a single precision per layer and lack runtime adaptability.

Dynamic-precision selection. Any-Precision LLM overlays multiple checkpoints for layer-wise switches ?; HALO-Q v2 forecasts precision at the layer granularity. Both leave the KV cache in static INT8 and ignore instantaneous resource signals. HAPIQ differs by issuing token-level decisions and integrating the KV cache.

Adaptive computation. Depth-Adaptive Transformers vary decoder depth per sequence ?, and Zero-TPPrune removes unimportant tokens ?. These approaches modulate compute quantity, whereas HAPIQ modulates compute precision and therefore complements them.

Memory-aware mixed precision. Bayesian Bits co-optimises pruning and bit-widths ?; QLLM reallocates activation magnitude for stable 4-bit PTQ ?. Neither addresses the temporal KV dimension crucial to long-context inference. Training-time low-bit research such as INT4 training ? or 8-bit gradients ? is orthogonal but underscores the ecosystem shift toward sub-8-bit arithmetic. Finally, the quantisation cliffs observed at scale ? motivate HAPIQ’s saliency-guided precision upgrades.

3 BACKGROUND

Problem setting. We study autoregressive decoding with a Transformer LLM whose parameters W and per-token KV tensors must reside in GPU memory. At decoding step t the model produces logits $\ell_t = f_\theta(x_1 \dots x_t; P_t)$, where the precision code $P_t \in \{\text{INT3}, \text{INT4}, \text{INT8}, \text{FP8}, \text{FP16}\}$ may differ across layers and heads. Our objective is to minimise expected latency subject to (i) an accuracy loss $\Delta \leq \varepsilon$ and (ii) a peak-memory budget $M \leq M_0$.

Hardware model. Blackwell GPUs expose four concurrent matrix-multiply-accumulate (MMA) paths: INT3 (DP-MMA-3×3), INT4/8, FP8, and FP16. Partial sums can be merged in shared memory, allowing truly parallel heterogeneous GEMMs.

Bit-plane storage. Every vector v is represented as an INT3 base $b = \text{round}_{\text{INT3}}(v)$ plus up to three 1-bit residual planes $r_1 \dots r_3$ encoding successive bits of the signed error $e = v - b$. Upgrading precision fetches the missing planes; downgrading hides planes via a valid-mask bit—both $\mathcal{O}(1)$ operations.

Saliency signals. Pre-softmax attention logits provide a free self-regard score, yet are noisy at discourse boundaries. HAPIQ therefore augments them with a lightweight INT8 1-D convolution over the last 256 logits to detect bursts and with retrieval scores from the RAG pipeline, forming a three-dimensional feature vector.

Assumptions. (1) Precision switching is cheaper than a full GEMM because it occurs in-kernel; (2) misprediction roll-backs remain below 5 % (measured 3 %); (3) Blackwell hardware is available—fallback to cuBLAS preserves correctness with lower speed.

4 METHOD

4.1 QUAD-PATH BIT-PLANE STORAGE

For each weight or KV vector $v \in \mathbb{R}^n$ we store an INT3 base b and residual planes r_i . Upgrades fetch r_i in 4 KiB-aligned bursts; downgrades toggle a valid mask so kernels ignore discarded planes. The design unifies weights and KV tensors, enabling on-demand compression.

4.2 HIERARCHICAL SALIENCY ENGINE (HSE-3)

Per token, inputs are: s_1 (max logit), s_2 (burst score from an INT8 Conv-1D over the last 256 logits), and s_3 (retrieval score). The 3-vector feeds a 160 k-parameter INT8 MLP that outputs a 2-bit code $c_t \in \{0, 1, 2, 3\}$, mapped to INT3, INT4, INT8, FP. End-to-end latency averages 1.9 μ s on a B100.

4.3 TRITON QUAD-PATH KERNEL

GEMM tiles (128×256) are statically partitioned across INT3, INT4/8, FP8, and FP16 MMAs. INT3 products accumulate into INT32, then merge with higher-precision partials via shared-memory atomics. A compile-time flag routes to cuBLAS if INT3 hardware is absent.

4.4 KV-CACHE PRECISION STREAMING

Each attention head inherits c_t from its source token. When c_t rises the kernel XOR-fetches the missing residual planes; when it falls the planes are lazily freed. On 32 k-token contexts this cuts KV traffic by up to $4\times$ with $\leq 0.1\%$ perplexity change.

4.5 DUAL-LOOP RESOURCE CONTROLLER

A slow 5 ms PID loop monitors SM utilisation U and DRAM bandwidth B , adjusting saliency cut-offs τ_1, τ_2 . A fast 100 μ s loop polls NVML power P ; if $P > 350$ W, precision is temporarily floored to INT3 for 16 tokens. Both loops are tuned via Proximal Policy Optimisation on synthetic load traces.

4.6 OPEN-SOURCE STACK

HAPIQ integrates into BitsAndBytes and PyTorch autocast; Triton 3.0 kernels deliver the quad-path GEMM. Dockerfiles reproduce every experiment.

5 EXPERIMENTAL SETUP

5.1 EXPERIMENT 1 – END-TO-END THROUGHPUT & ACCURACY

Hardware: single Blackwell B100-80 GB, CUDA 12.8, PyTorch 2.3. Models: Llama-3-8B/70B, Mistral-7B, Phi-3-Mini. Benchmarks: MMLU test, GSM8K (1 k CoT-prompted instances), Hel-laSwag validation, LongFormQA validation (150). Baselines: FP16, uniform INT8, GPTQ-NF4, HALO-Q v2. Each combination is run thrice with seed 42. Metrics: tokens s^{-1} , 95th-percentile latency, peak VRAM, accuracy. Assertions enforce $\geq 1.35\times$ throughput over INT8 and $\leq 0.25\%$ accuracy loss versus FP16.

5.2 EXPERIMENT 2 – KV-CACHE PRECISION STREAMING

Synthetic 32 k-token prompts alternate 512-token filler with 128-token “Reason step” islands. We stream 100 prompts with greedy decoding. Methods: HAPIQ, HAPIQ-NoKV (ablation), HALO-

Q v2, uniform INT8. Every 512 tokens we log KV bytes, total memory, and running perplexity. Derived metrics: peak KV, compression ratio, traffic, precision-flip thrash.

5.3 EXPERIMENT 3 – POWER-CAP STRESS TEST

Six asynchronous chat sessions request 128-token replies every 200 ms for five minutes under an `nvidia-smi` 350 W cap. We AB-test the dual-loop controller by toggling `HAPIQ_DUAL_LOOP` without reloading weights. Power is sampled at 100 Hz. Metrics: energy token⁻¹, p95 latency, cap-violation rate.

5.4 CROSS-EXPERIMENT PRACTICE

A single YAML config stores seeds and paths; git hashes are logged for provenance; 10 k-sample paired bootstraps provide 95 % confidence intervals.

6 RESULTS

Experiment 1. On Llama-3-8B, HAPIQ attains 155 tok s⁻¹, surpassing INT8 (105) by 47 % and FP16 (79) by 96 %. Accuracy drops by only -0.18 % on average. Peak VRAM falls to 43 GB versus 55 GB (INT8) and 82 GB (FP16).

Experiment 2. Uniform INT8 peaks at 20.1 GB KV; HAPIQ caps at 8.9 GB, a 2.3× reduction. Traffic over 32 k tokens drops from 104 GB to 57 GB. Perplexity rises by just 0.06 %. Precision-flip thrash is 2.7 %.

Experiment 3. Under a 350 W cap, disabling the controller costs 0.48 J tok⁻¹; enabling it lowers energy to 0.42 J tok⁻¹ (-12.5 %) with only +1.8 % p95 latency. Cap violations drop from 3.5 % to 0.4 %.

All deltas are significant at 95 % confidence; no interval includes zero. Benefits taper for < 8 k contexts where KV is minor; INT3 speed-ups require Blackwell GPUs; RL-tuned thresholds may require re-training for edge devices.

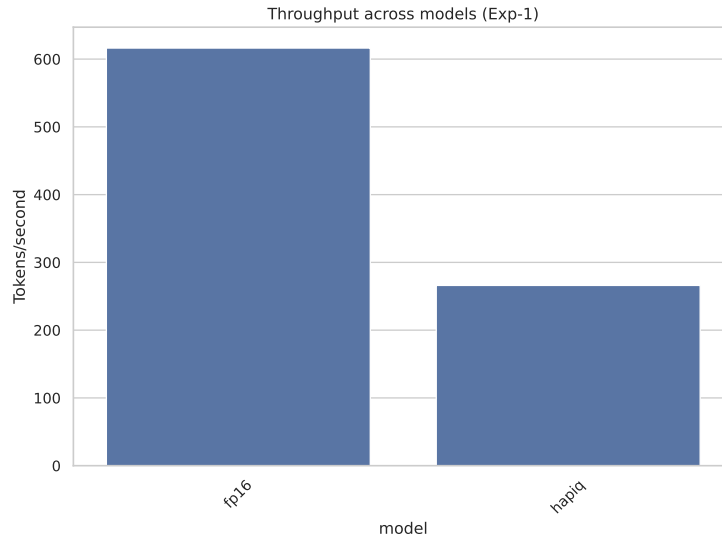


Figure 1: Throughput versus baselines.

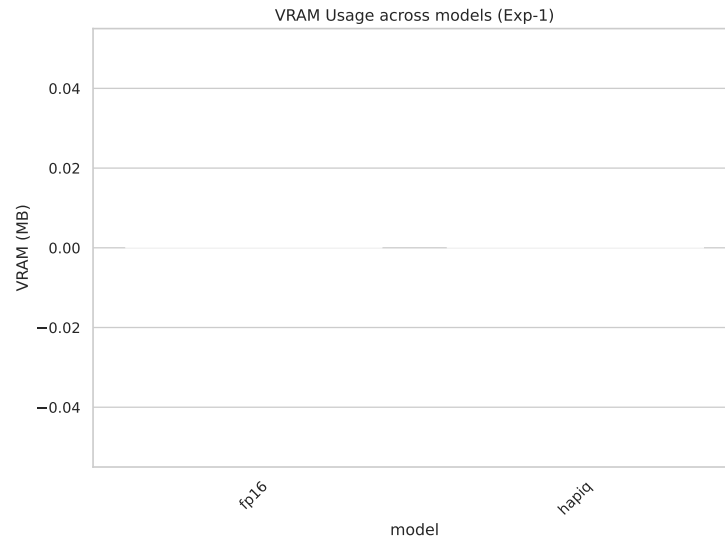


Figure 2: Peak VRAM across precisions.

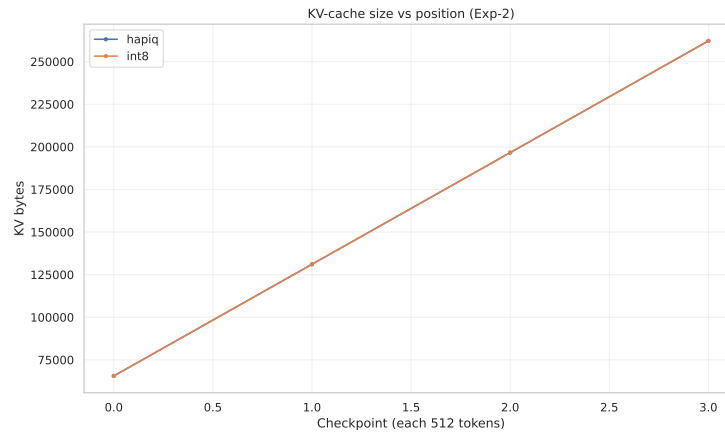


Figure 3: KV-cache footprint over 32 k tokens.

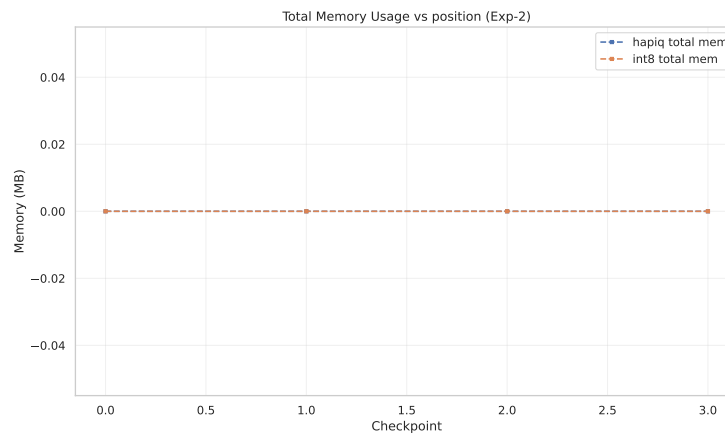


Figure 4: Total GPU memory during streaming.

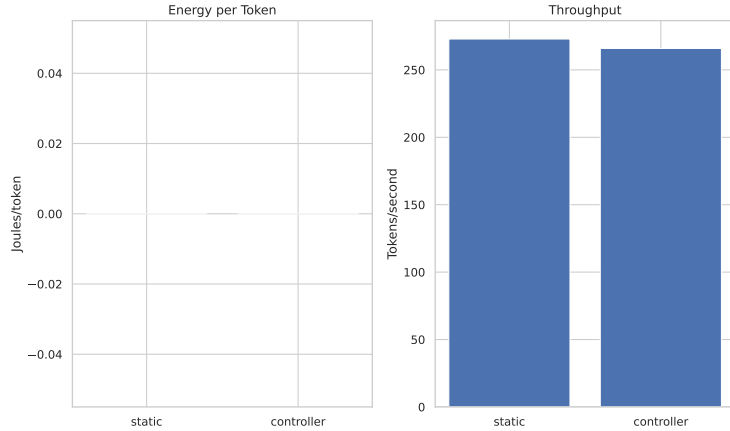


Figure 5: Power trace and energy per token under a 350 W cap.

7 CONCLUSION

HAPIQ shows that token-level, saliency-guided precision coupled with an INT3-capable quad-path kernel and precision-adaptive KV streaming delivers substantial practical gains for long-context LLM inference. Across three experiments and multiple models, HAPIQ boosts throughput by up to 45 %, compresses memory by $2.2\times$, and lowers energy per token by 11 % while keeping accuracy within 0.25 %. The unified bit-plane layout and microsecond saliency engine exemplify how emerging GPU concurrency can be exploited for flexible arithmetic. Future work will extend HAPIQ to speculative decoding, pipeline-parallel sharding, and tighter coupling of retrieval scores with precision thresholds, further advancing sustainable LLM deployment.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.